

Hexabid User Guide

Table of Contents

1. Cel dokumentu	1
2. Logowanie	1
3. Podstawowy przepływ	2
3.1. 1. Pobranie aukcji	2
3.2. 2. Utworzenie aukcji	2
3.3. 3. Subskrypcja aukcji	2
3.4. 4. Złożenie oferty	3
4. Zmiana prowadzącego	3
5. Zakończenie aukcji	4
5.1. Aukcja z rozstrzygnięciem	4
5.2. Aukcja bez rozstrzygnięcia	4
6. Obsługa błędów	4
6.1. Błędy biznesowe przy licytacji	4
6.2. Konflikt współbieżności	5
6.3. Błędy uwierzytelnienia	5
7. Zalecany przebieg po błędzie	5
8. Uwagi integracyjne	5

1. Cel dokumentu

Ten dokument opisuje podstawowy sposób korzystania z systemu **Hexabid** z perspektywy użytkownika biznesowego i integratora klienta.

2. Logowanie

System wymaga uwierzytelnienia przed:

- utworzeniem aukcji przez REST
- składaniem ofert przez WebSocket

Dostępni dostawcy:

- Google
- GitHub

Tożsamość użytkownika jest pobierana z sesji OAuth2/OpenID Connect. Klient nie przekazuje **sellerId** w REST ani **bidderId** w komunikatach WebSocket.

3. Podstawowy przepływ

3.1. 1. Pobranie aukcji

Klient pobiera szczegóły aukcji przez REST:

```
GET /api/auctions/{auctionId}
```

Możliwe wyniki:

- **200 OK** z widokiem aukcji
- **404 Not Found**, gdy aukcja nie istnieje

3.2. 2. Utworzenie aukcji

Uwierzytelniony sprzedający tworzy aukcję przez REST:

```
POST /api/auctions
Content-Type: application/json

{
  "title": "Komiks z limitowanej serii",
  "startingPrice": {
    "amount": "100.00",
    "currency": "PLN"
  },
  "endsAt": "2026-03-10T12:00:00Z"
}
```

Możliwe wyniki:

- **201 Created**, gdy aukcja została utworzona
- **400 Bad Request**, gdy czas zakończenia nie jest w przyszłości
- **400 Bad Request**, gdy sprzedający nie przeszedł weryfikacji KYC
- **401 Unauthorized**, gdy użytkownik nie jest zalogowany

3.3. 3. Subskrypcja aukcji

Klient łączy się z WebSocket STOMP pod:

```
ws://localhost:8080/ws-auctions
```

Kanały:

- `/topic/auctions/{auctionId}/bids` - zaakceptowane oferty
- `/topic/auctions/{auctionId}/events` - zdarzenia biznesowe aukcji
- `/topic/auctions/{auctionId}/errors` - odrzucone oferty i błędy biznesowe

3.4. 4. Złożenie oferty

Klient wysyła komunikat STOMP na:

```
/app/auctions/{auctionId}/bids
```

Payload:

```
{
  "amount": "130.00",
  "currency": "PLN"
}
```

Możliwe wyniki:

- na `/topic/auctions/{auctionId}/bids` trafia potwierdzenie zaakceptowanej oferty
- na `/topic/auctions/{auctionId}/events` trafia zdarzenie `AUCTION_LEADER_CHANGED`
- na `/topic/auctions/{auctionId}/errors` trafia komunikat odrzucenia, jeśli oferta nie została przyjęta

4. Zmiana prowadzącego

Jeżeli nowa oferta przebija poprzednią, system publikuje zdarzenie:

- `AUCTION_LEADER_CHANGED`

Payload zawiera:

- poprzedniego lidera, jeśli istniał
- nowego lidera
- nową cenę prowadzącą
- walutę

To zdarzenie jest emitowane równolegle do:

- WebSocket dla klientów w czasie rzeczywistym
- Kafki dla integracji asynchronicznych

5. Zakończenie aukcji

Scheduler okresowo zamyka wygasłe aukcje.

5.1. Aukcja z rozstrzygnięciem

Jeżeli przynajmniej jedna ważna oferta została zapisana, system publikuje:

- `AUCTION_WON`

Payload zawiera:

- zwycięzcę
- cenę końcową
- walutę

5.2. Aukcja bez rozstrzygnięcia

Jeżeli nikt nie wziął udziału w aukcji, system publikuje:

- `AUCTION_CLOSED_WITHOUT_WINNER`

To oznacza, że aukcja została zamknięta, ale nie ma zwycięzcy i nie doszło do sprzedaży.

6. Obsługa błędów

6.1. Błędy biznesowe przy licytacji

System nie zwraca surowych wyjątków do klienta WebSocket. Zamiast tego publikuje jawny wynik odrzucenia z kodem biznesowym.

Przykładowe przyczyny:

- `AUCTION_NOT_FOUND`
- `BIDDER_NOT_VERIFIED`
- `SELLER_CANNOT_BID_ON_OWN_AUCTION`
- `BID_AMOUNT_TOO_LOW`
- `AUCTION_ALREADY_CLOSED`
- `AUCTION_ALREADY_EXPIRED`
- `CURRENCY_MISMATCH`

- `CONCURRENT_MODIFICATION`

6.2. Konflikt współbieżności

System używa optimistic locking przy zapisie aukcji.

Skutek:

- jeżeli dwie oferty próbują jednocześnie zmienić ten sam stan aukcji, jedna może zostać odrzucona jako `CONCURRENT_MODIFICATION`
- system nie traci spójności danych
- klient może ponowić pobranie stanu aukcji i wysłać nową ofertę

Job zamykający aukcje działa stabilnie także przy konflikcie:

- konflikt jest liczony i pomijany
- pozostałe aukcje są dalej przetwarzane

6.3. Błędy uwierzytelnienia

Jeżeli użytkownik nie jest zalogowany:

- REST zwraca `401 Unauthorized`
- WebSocket publikuje odrzucenie z kodem `UNAUTHENTICATED`

7. Zalecany przebieg po błędzie

1. Pobierz aktualny stan aukcji przez REST.
2. Zweryfikuj, czy aukcja nadal jest otwarta.
3. Dla konfliktu współbieżności lub przebicia oferty wyślij nową ofertę z wyższą kwotą.
4. Dla błędów KYC dokończ weryfikację użytkownika poza systemem aukcyjnym.

8. Uwagi integracyjne

- identyfikator użytkownika w domenie jest wyprowadzany z dostawcy logowania i jego identyfikatora zewnętrznego
- klient nie powinien próbować nadawać własnej tożsamości w payloadach
- kanał `/events` służy do faktów biznesowych, a `/errors` do jawnych odrzuceń operacji